

CONTENTS

EduOS Cameroon — Technology Stack and Architecture Decision Record	
0. Selection principles (“test of time” defined)	
ADR-01 — System topology: Modular monolith with DDD boundaries, evolving to services	
ADR-02 — Database: PostgreSQL 16+, single source of truth	
ADR-03 — Backend language & framework: PHP 8.3+ / Laravel 11, with strict module architecture	
ADR-04 — Mobile: Flutter with SQLite (offline-first client)	
ADR-05 — Platform services (all open source)	
ADR-06 — Domain-Driven Design: the boundaries are the architecture	
ADR-07 — API-first, and the API as a product	
ADR-08 — What we deliberately did NOT choose	
ADR-09 — Offline sync engine foundation (OPEN — must close before build contract)	
ADR-10 — Offline authentication (OPEN — must close before build contract)	
ADR-11 — School device strategy: tiered, with rugged tablets and travel-to-sync for remote schools	
Known gaps and open decisions (honest register)	
Summary stack card	

EduOS Cameroon — Technology Stack and Architecture Decision Record

Document ID	EDUOS-TSA-001
Version	1.0
Developer	Opesware Technologies · Douala, Cameroon · www.opesware.com · eudos@opesware.com · +237 670 41 62 38
Format	Architecture Decision Records (ADR): each decision states context, options, choice, and consequences
Governing constraints	FRS NFRs (offline-first, 30M copies/300M events, PostgreSQL, no lock-in), Risk Register (R2 connectivity, R4 vendor lock-in, R5 recurrent cost, R11 key-person), Baseline Annex §5 (26% rural electricity, 2G/3G pockets)

0. Selection principles (“test of time” defined)

A national platform outlives every framework hype cycle, every ministry cabinet, and probably Opesware’s current team. “Stands the test of time” therefore means, in priority order:

1. **Maintainable by the talent pool that exists in Cameroon** — not the talent pool of San Francisco. A stack is only sustainable if MINEDUB/MINESEC’s national technical team (BUD §3.7) can hire for it in Douala and Yaoundé in 2035.
2. **Open source, no per-seat/per-core licences** — the recurrent budget (1.22 bn FCFA/yr) must buy people and hosting, not licence renewals; sovereignty demands source access (Risk R4).
3. **Boring and proven at national scale** — every component ≥ 10 years old with a massive installed base and multiple support vendors.
4. **Data outlives code** — the database schema and the API contracts are the 20-year assets; any application layer must be rewritable against them.

ADR-01 — System topology: Modular monolith with DDD boundaries, evolving to services

Context. The vision volumes (Chapter 27) describe a 22-service microservice landscape. Microservices solve organizational scaling (many teams deploying independently) at the price of distributed-systems complexity: network partitions, distributed tracing, eventual consistency everywhere, and an ops burden measured in dedicated platform engineers. Phase I is built by **one team** (Opesware + national counterparts) for a system whose hardest problem is *offline sync*, not horizontal service scaling: 70 warehouses and 18,500 schools syncing daily is modest central load (NFR-NTR-03: 200 sync sessions/minute peak).

Decision. Build a **modular monolith**: one deployable backend whose internal modules are the DDD bounded contexts (ADR-06), communicating only through in-process interfaces and domain events — never by reaching into each other's tables. Three components are separate processes from day one because their runtime profile genuinely differs: the **sync gateway** (long-lived connections, burst absorption), the **notification worker**, and the **reporting/read replica**. Chapter 27's service landscape is retained as the **target decomposition map**: any module can be extracted into a service later because the boundaries are already contractual.

Why this stands the test of time. The failure mode that kills national systems is not “the monolith didn't scale” — it is “nobody left understands the distributed system.” A modular monolith is operable by a 4-person national team (Risk R11); premature microservices would consume the entire recurrent budget in DevOps. Extraction remains a refactoring, not a rewrite, because module boundaries = context boundaries.

ADR-02 — Database: PostgreSQL 16+, single source of truth

Decision. PostgreSQL for all transactional data (already normative, NFR-NTR-09/NFR-NWD-05). Partitioning for PassportEvent/CustodyEvent by school-year (NFR-NTR-01). Read replica for reports and the public catalogue. **No polyglot persistence in Phase I** — one database technology the national team masters deeply beats four mastered shallowly.

Why. PostgreSQL is 30 years old, fully open source, runs on national data-centre hardware, handles 300M-row event tables comfortably with partitioning, and offers JSONB (event payloads), PostGIS (school mapping, FR-NSR-08, redistribution proximity FR-NWD-11), and logical replication (DR, RPO 24h) — the entire storage requirement in one boring engine.

ADR-03 — Backend language & framework: PHP 8.3+ / Laravel 11, with strict module architecture

Options weighed. Java/Spring Boot (donor-classic, strong typing, heavier ops + smaller local pool), Node/NestJS (large pool, TypeScript safety, weaker long-term backward-compat record), PHP/Laravel (largest Central-African talent pool, Opesware's core competency, mature ecosystem), Python/Django (pool exists but smaller for enterprise systems locally).

Decision. Laravel (PHP 8.3+) as the backend framework, under discipline: modules per bounded context (e.g., `Modules/Registry`, `Modules/Custody`), no cross-module Eloquent access, domain events on the internal bus, static analysis (PHPStan level 8) and architecture tests (Deptrac) enforcing boundaries in CI.

Why. Criterion 1 dominates: PHP is the most hireable enterprise skill in Cameroon's market, and the developer (Opesware Technologies, Douala) builds on it — meaning the people who wrote the system and the people who will maintain it come from the same pool. PHP 8 with strict types + PHPStan closes most of the type-safety gap to Java; Laravel has a 13-year backward-compatible upgrade record, LTS discipline, and first-class queue/job infrastructure for the sync workload. Wikipedia, Slack's backend origins, and half the world's government portals demonstrate PHP's longevity at national scale. **Consequence accepted:** CPU-heavy analytics do not belong in PHP — they live in the database (SQL/materialized views) and, if ever needed, a dedicated worker (ADR-01 extraction path).

ADR-04 — Mobile: Flutter with SQLite (offline-first client)

Decision. One **Flutter** codebase for the Android school/warehouse app; local store **SQLite** (drift), camera QR scanning, background sync against the sync gateway implementing FRS-NTR §9 (UUIDv7 events, resumable chunked push, quarantine rules).

Why. The 90-day-offline requirement (NFR-NTR-05) makes the mobile app a full local system, not a thin client — it needs a real embedded database and full business rules locally. Flutter delivers native performance on 2 GB-RAM Android 10 devices, one codebase for future iOS/desktop, and Google-backed longevity; SQLite is the most deployed database on earth and will outlive everything else in this document. Web fallback (responsive Laravel/Inertia views) covers private schools on BYOD (BUD §5.2).

ADR-05 — Platform services (all open source)

Concern	Choice	Why it lasts
Identity & access (IAM)	Keycloak (OIDC/OAuth2)	The FRS mandates central IAM with role claims; Keycloak is the de-facto open-source standard, Red Hat-backed, self-hostable in-country
API gateway	Apache APISIX (or Kong OSS)	Rate limiting, API keys, quotas per consumer — required for API-as-a-product (§ below); both are CNCF-ecosystem, no licence cost
Async/event bus	Redis + Laravel queues Phase I; RabbitMQ when extraction begins	Don't run Kafka for 200 msgs/minute; the outbox pattern (FR-NWD-DM-03) works against any broker
Object storage	MinIO (S3 API)	Condition photos, label PDFs, report exports; S3 API = zero lock-in, self-hosted in the national DC
Observability	Prometheus + Grafana + Loki	SYS-1..4 M&E indicators come straight from here; the open-source observability standard
Deployment	Docker Compose → K3s	Containers from day one; plain Compose for pilot simplicity, lightweight Kubernetes (K3s) at national rollout when HA matters — never a cloud-proprietary orchestrator
Hosting	National data centre primary + DR (BUD §3.2), FCFA-denominated	Sovereignty + Risk R10 currency exposure
CI/CD & source	GitLab CE self-hosted in-country; source escrow per Risk R4	The repository is a national asset; quarterly escrow deposits are contractual

Everything above is open source with multiple commercial support options — no single vendor, including Opesware, can hold the platform hostage (this protects Opesware too: it makes the sovereignty conversation with financiers winnable).

ADR-06 — Domain-Driven Design: the boundaries are the architecture

DDD is not a buzzword here; it is how the FRS documents were already written. The **ubiquitous language** is fixed and appears identically in requirements, code, and UI: *Title, Edition, Batch, Copy, Passport, Custody, Shipment, Allocation, Verification Campaign*. Rules:

Bounded contexts (= modules = future services):

Context	Owns	Key aggregates (consistency boundaries)
Curriculum & Catalogue	Titles, editions, approval workflow	Title (with Editions)
School Registry	Schools, hierarchy, enrolment returns	School (with StatusEvents, EnrolmentReturns)
Custody & Logistics (NWIDMS)	Stock ledger, shipments, discrepancies	Shipment (with CustodyEvents); StockRecord transactions
Asset Passport (NTR runtime)	Copies, passport events, verification	Copy (with PassportEvent hash chain)
School Operations	Assignments, returns, condition	StudentAssignment
Identity & Access	Users, roles, devices	delegated to Keycloak
Analytics & Reporting	Read models only — no writes, ever	projections

Context-mapping rules. Contexts integrate via **domain events** (BatchRegistered, ShipmentDispatched, CopyAssigned...) and published contracts — never shared tables. Analytics is strictly downstream (conformist consumer of events). External ministry systems (legacy EMIS, payroll) connect through **anti-corruption layers** so their models never leak in. The aggregates encode the invariants that matter nationally: a Copy's passport chain is append-only and hash-linked *inside one aggregate*; a Shipment cannot close with unexplained variance *inside one aggregate* — which is why these rules survive refactors.

Why DDD is the longevity strategy. Frameworks will be replaced (ADR-03 might be re-decided in 2035); the *domain model* — what a Copy is, what custody means, what closes a shipment — is permanent. DDD puts the permanent thing at the centre and treats the technology as

replaceable skin, which is exactly the property a 20-year national system needs.

ADR-07 — API-first, and the API as a product

API-first (engineering practice). For every module, the **OpenAPI 3.1 contract is authored and reviewed before implementation** (deliverables D-NTR-API, D-NSR-API, D-NWD-API, generated from FRS §7 sections which govern on conflict). Consequences: contract tests (Schemathesis) run in CI against every build; mobile and web teams develop against generated mocks in parallel with the backend; breaking changes are structurally impossible to ship silently (`/api/v1` guaranteed ≥ 24 months after v2, FR-NTR-API-02); every endpoint carries idempotency and pagination rules from the FRS.

API as a product (institutional strategy). The registries are **national digital public infrastructure**, and their APIs are the product other actors build on:

- **Named consumers with tiers:** internal modules (full), other government systems via anti-corruption gateways (partner tier), publishers/logistics contractors (contractual tier: batch registration, shipment tracking), researchers and civil society (public tier: the school directory FR-NSR-05 and textbook catalogue FR-NTR-13 — which, per the carte scolaire finding in BDA §6, would be **Cameroon's first open machine-readable school dataset**).
- **Product management:** a developer portal (docs, sandbox, API keys via the gateway), published SLAs per tier, versioned changelogs, quota/rate policies, and a designated API product owner inside the national team. Uptake metrics (external consumers, calls/month) become platform KPIs.
- **Why this matters to financiers:** APIs-as-product converts a ministry IT project into reusable national infrastructure — the same argument that funded India's DPI stack — and creates the local-ecosystem benefits (Douala/Yaoundé startups building on the public APIs) that development partners explicitly fund.

ADR-08 — What we deliberately did NOT choose

Rejected	Reason
Microservices day one	Ops burden > national team capacity; see ADR-01
Proprietary cloud services (managed queues, serverless, vendor AI)	Sovereignty, FCFA budgeting, NFR-NTR-09 exit-path requirement
Blockchain for passports	The hash-chained append-only event log (FR-NTR-DM-02) delivers tamper-evidence without consensus overhead, at 1/100th the complexity
NoSQL primary store	The domain is deeply relational (registries, ledgers); JSONB covers the flexible parts
React Native / native Java+Swift	Two codebases or a weaker offline story vs Flutter+SQLite
Kafka	200 events/minute does not need a distributed log; revisit only at extraction stage

ADR-09 — Offline sync engine foundation (OPEN — must close before build contract)

Context. The sync protocol (FRS-NTR §9: delta pull, resumable chunked push, conflict quarantine) is the highest-risk custom component in the platform; bespoke sync engines are notoriously bug-prone. The stack gives local storage (SQLite) but not synchronization.

Decision required. Evaluate build-vs-adopt against PowerSync, ElectricSQL, and Couchbase Lite/Sync Gateway before the build contract is signed; if the verdict is “build custom,” Phase I SHALL include a sync-engine prototype gate (30-day offline soak + reconciliation drill) before any dependent module development. **Status: OPEN — a named Phase-0 deliverable.**

ADR-10 — Offline authentication (OPEN — must close before build contract)

Context. Keycloak issues OIDC tokens, but a head teacher must authenticate on day 60 of a 90-day offline stretch, and every mutation must remain user-attributable (NFR-NTR-07). Long-lived bearer tokens contradict security policy.

Direction. Device-bound offline credentials: device certificate (enrolment, FRS-NTR §9.1) + local PIN-unlocked user credentials with offline role cache, deferred revocation applied at next sync, and all offline actions signed with the device key. Exact scheme (and its ANTIC compliance review) is a Phase-0 security design deliverable. **Status: OPEN.**

ADR-11 — School device strategy: tiered, with rugged tablets and travel-to-sync for remote schools

Context. School conditions split sharply (NSR `accessibility_class` + `connectivity` + `grid_power` fields exist precisely for this): connected town schools vs. remote schools with no/2G signal and no grid power (rural electrification 26%, BDA §5).

Decision. Two device tiers, allocated from School Registry data:

Tier	Schools	Device class	Sync pattern
Standard	URBAN / RURAL_ROAD with ≥3G	Mainstream Android 10+ smartphone/tablet, rugged case	Opportunistic background sync (daily/weekly)
Rugged	RURAL_SEASONAL / REMOTE, poor or no signal, off-grid	Rugged tablet of the Blackview Active 8 Pro class or higher: MIL-STD- 810H / IP68-IP69K rated, ≥10" screen, very large bat- tery (~22,000 mAh class → weeks of duty-cycle use per charge), Android 13+, ≥8 GB RAM	Travel-to-sync: the school operates fully offline — re- ceipts, assignments, returns, condition, verification cam- paigns all executed locally — and the device is carried to a network location (sub-divi- sional inspectorate, division office sync point, or market- town coverage area) weekly to monthly for a batched sync session

Why this works without any architecture change. The platform was designed for exactly this: NFR-NTR-05 requires 90 consecutive offline days, sync push is chunked and resumable (survives a flaky 2G/3G session), events carry device time + UUIDs so late-arriving batches reorder correctly, and the quarantine rules (§9.4) absorb whatever conflicts a month of offline operation produces. Travel-to-sync is the *designed-for* case, not a workaround. Division sync points (Risk R2 mitigation, already budgeted) double as charging stations.

Consequences.

- The ~22,000 mAh battery class materially reduces solar-kit dependency: a device charged at the weekly/monthly sync trip can run its duty cycle between trips, so solar kits are re-targeted to the deepest-remote subset only (est. 3,500 rather than 5,500). The rugged-tier unit premium (~140,000 vs 92,000 FCFA) is largely offset by that reduction; the residual net increase (≈ 0.1 bn FCFA) sits within the budget's 10% contingency (BUD §2 line 9). Final tier counts come from the NSR data-cleaning campaign — another reason it precedes device procurement (FRS-NSR §9).
- SYS-2 (M&E) stays honest: “ ≥ 1 sync in last 90 days” accommodates the monthly-sync tail; the per-school expected-sync-interval is configured from its tier so overdue alerts don't false-positive on remote schools.
- Procurement SHALL specify the *class* (MIL-STD-810H/IP68, battery $\geq 15,000$ mAh, Android 13+, 3-year parts availability), not a single brand, with Blackview Active 8 Pro named as the reference device — keeping competitive bidding valid (≤ 3 models per wave, Risk R14).

Known gaps and open decisions (honest register)

Beyond ADR-09/10 (blocking, Phase-0) — named before procurement so no vendor can fail expensively on an ambiguity:

#	Gap	Resolution path
G1	Sync-gateway runtime: PHP-FPM is wrong for long-lived burst connections	Laravel Octane (Swoole) or a small Go/Node service — decided at ADR-09 evaluation; never plain PHP-FPM
G2	No SMS/USSD fallback channel for the disconnected tail	Phase-II evaluation: SMS delivery-confirmation codes via local aggregator for schools that cannot make even monthly sync trips
G3	MDM for 18,500 devices unnamed	Headwind MDM (open source) or Android Enterprise; enrolment integrated with device certificates (ADR-10); remote wipe per Risk R9
G4	BI/dashboard tool unnamed	Metabase (or Superset) over the read replica; ministry dashboards are configurations, not custom code
G5	Backup tooling unnamed for RPO 24h/RTO 72h	pgBackRest + streaming replica + offsite copies to DR site; restore drill quarterly
G6	Security ops: secrets, WAF, SIEM, scanning, ANTIC compliance unaddressed	OpenBao (Vault fork) for secrets; ModSecurity/Coraza WAF at gateway; Wazuh SIEM; Trivy scanning in CI; ANTIC review as Phase-0 deliverable alongside the DPIA (Risk R15)
G7	Offline map tiles pipeline unnamed (FR-NSR-08)	Protomaps single-file tile packs shipped with the app
G8	Load-testing & real-device testing unnamed	k6 for NFR load numbers; a small physical device lab of the actual procured models (incl. the rugged reference device)
G9	Flutter longevity = a bet on Google	Accepted: all logic lives behind the API and SQLite; the UI layer is rewritable without touching the domain
G10	K3s assumes ops skills that don't exist yet	Phase I runs plain Docker Compose; K3s adoption gated on the national team passing an ops-readiness assessment, with training budgeted (BUD §3.4)
G11	300M-event long-horizon analytics will outgrow the read replica	Deferred deliberately; columnar store (ClickHouse or DuckDB over exported Parquet) evaluated at Phase IV, fed by the same domain events
G12	Module-boundary discipline is procedural, not technical	CI boundary checks (Deptrac) made contractual: merge-blocking, verified at phase gates

Summary stack card

PostgreSQL 16 · Laravel/PHP 8.3 modular monolith (DDD bounded contexts) · Flutter+SQLite offline-first mobile · Keycloak IAM · APISIX gateway · Redis queues · MinIO · Prometheus/Grafana · Docker→K3s · self-hosted GitLab · national DC hosting — 100% open source, API-first with OpenAPI-governed contracts, APIs operated as national digital public infrastructure.

Developed by **Opesware Technologies**, Douala, Cameroon — www.opesware.com · eudos@opesware.com · +237 670 41 62 38 — under the source-escrow, knowledge-transfer, and national-team co-development obligations of Risk R4 and Budget §3.7.